

COMP90038
Algorithms and Complexity

Cezary Kaliszyk
(thanks to Harald Søndergaard)

Lecture 21

Huffman Encoding for Data Compression

Announcements

- Please fill Student Learning Surveys

Last lecture: Greedy algorithms

Prim's algorithm

minimum spanning tree

Dijkstra's algorithm

(single-source) shortest paths

Data Compression

What do we want compression to do?

Data Compression

What do we want compression to do?

- Most computer files contain much redundancy
- Compression is used to store files in less space.
- Savings of 50%–90% are common (depending on file kind)
- Savings in space mean savings in time for file transmission.
- Savings in space and time mean savings in energy usage.



Towards Compression: Ideas

Towards Compression: Ideas

AAAAAAAAAACCCCCCCCCC

Run-Length Encoding

Long runs of repeated characters

count the runs:

AAAABBBAABBBBBCCCCCCCCDABCBAAABBBBCCCD

can then be encoded as

4A3BAA5B8CDABCB3A4B3CD

Useful?

- For English text this is not very good
- For some binary files it can be effective

Run-Length Encoding

```
000000000000000000000000000011111111111111000000000
000000000000000000000000000011111111111111110000000
000000000000000000000000000011111111111111111110000
000000000000000000000000000011111111111111111111000
000000000000000000000000000011111111111111111111110
000000000000000000000000000011111111111111111111111
000000000000000000000000000011111111111111111111111
000000000000000000000000000011111111111111111111111
```

Variable-Length Encoding

Variable-Length Encoding

Idea

Assign characters codes in a way that common characters have short codes

Example

- “E” could have code 0
- No other character code can start with 0.

Property

- no character’s code should be a prefix of some other character’s code
- (unless we use separators, which would take up space)

Variable-Length Encoding

Suppose we count symbols and find these numbers of occurrences:

Symbol	Weight
B	4
D	5
G	10
F	12
C	14
E	27
A	28

Here are some sensible codes that we may use for symbols:

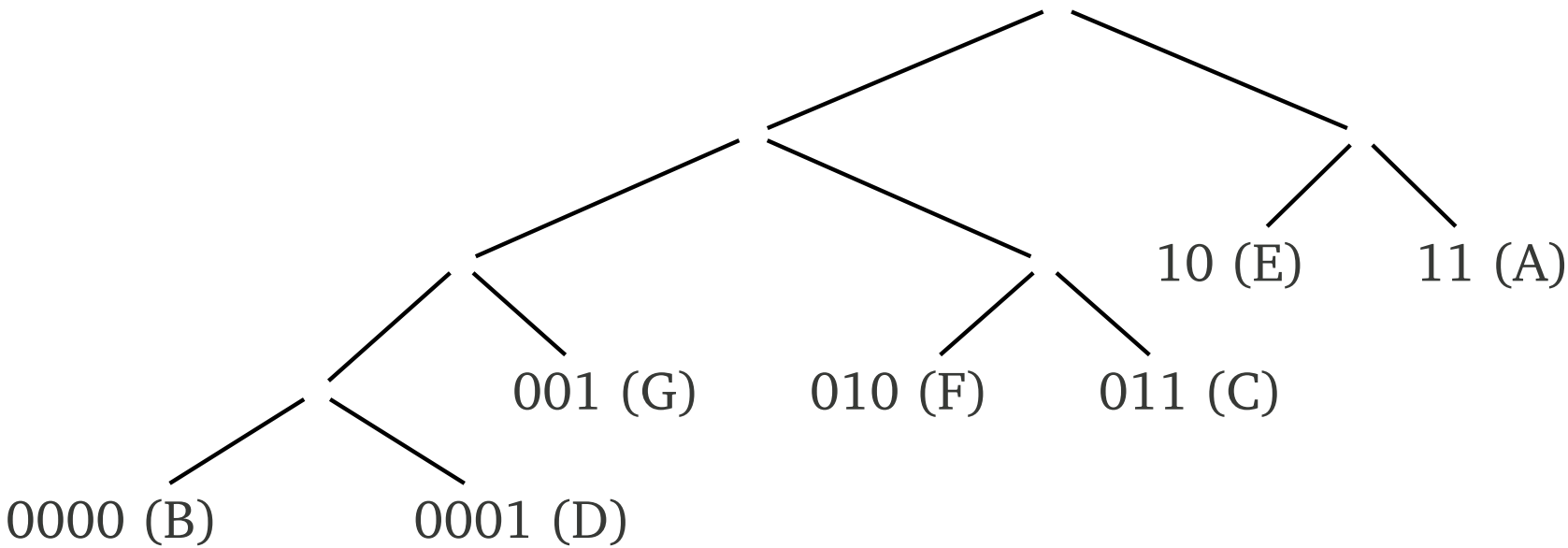
Symbol	Code
A	11
B	0000
C	011
D	0001
E	10
F	010
G	001

Tries for Variable-Length Encoding

A trie is a binary tree for search applications.

To search for a key we look at individual bits of a key and descend to the left whenever a bit is 0, to the right whenever it is 1.

Using a trie to determine codes means that no code will be the prefix of another!



Variable-Length Encoding

With the codes given by the trie, we can represent

FACE

in just 10 bits:

$$\underbrace{010}_{\text{F}}\ \underbrace{11}_{\text{A}}\ \underbrace{011}_{\text{C}}\ \underbrace{10}_{\text{E}}$$

If we were to assign 3 bits per character, FACE would take 12 bits.

Encoding a Message

We shall shortly see how to design the codes for symbols, taking symbol frequencies into account.

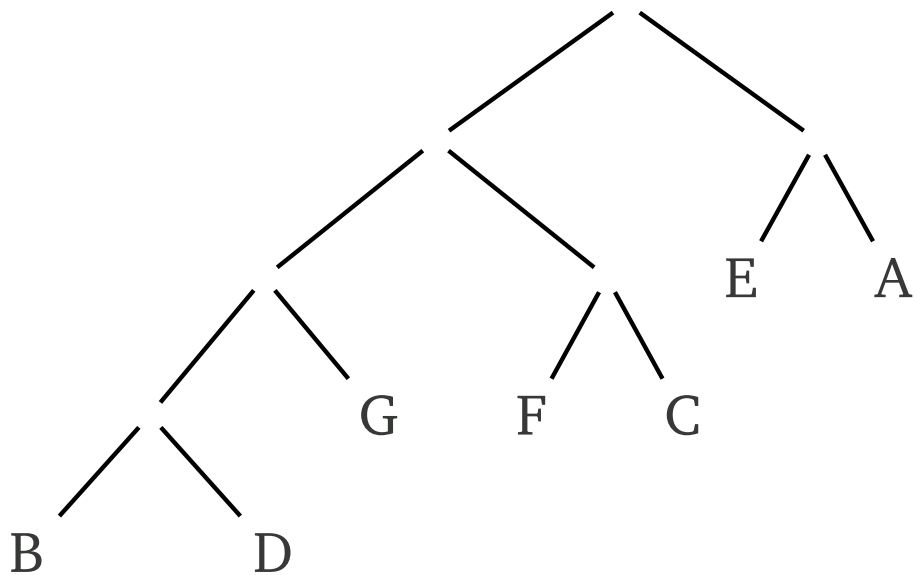
Once we have a table of codes, encoding is straightforward.

For example, to encode ‘BAGGED’, simply concatenate the codes for B, A, G, G, E and D:

0000110010011100001

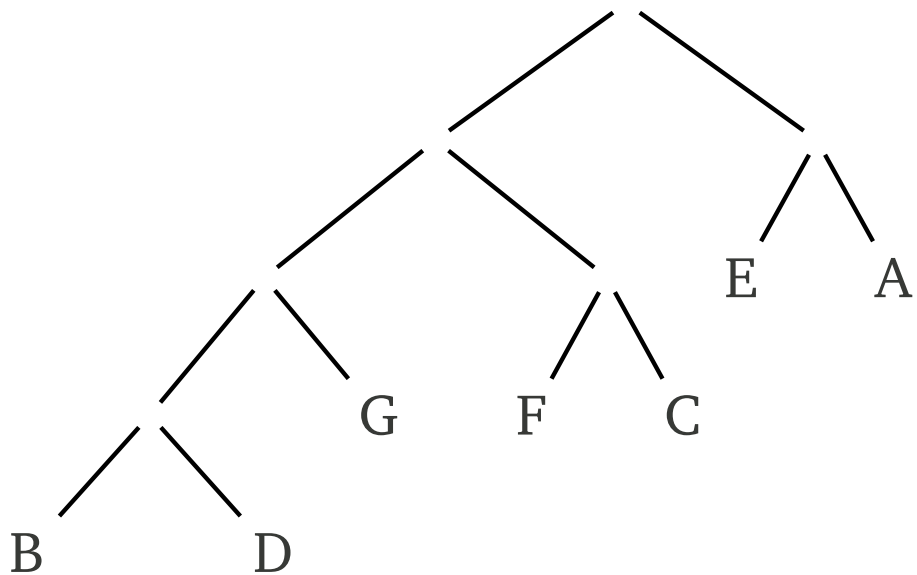
A	11
B	0000
C	011
D	0001
E	10
F	010
G	001

Decoding a Message



Decoding a Message

To decode 000011001001100001, use the trie, repeatedly starting from the root, and printing each symbol found as a leaf.



Huffman Encoding: Choosing the Codes

Huffman Encoding: Choosing the Codes

Frequencies of letters

- We know them for common English text fairly well
- We can count all characters as a first step

How to assign codes to the characters once we know their frequencies?

Huffman Encoding: Choosing the Codes

Frequencies of letters

- We know them for common English text fairly well
- We can count all characters as a first step

How to assign codes to the characters once we know their frequencies?

Huffman's algorithm (1952)

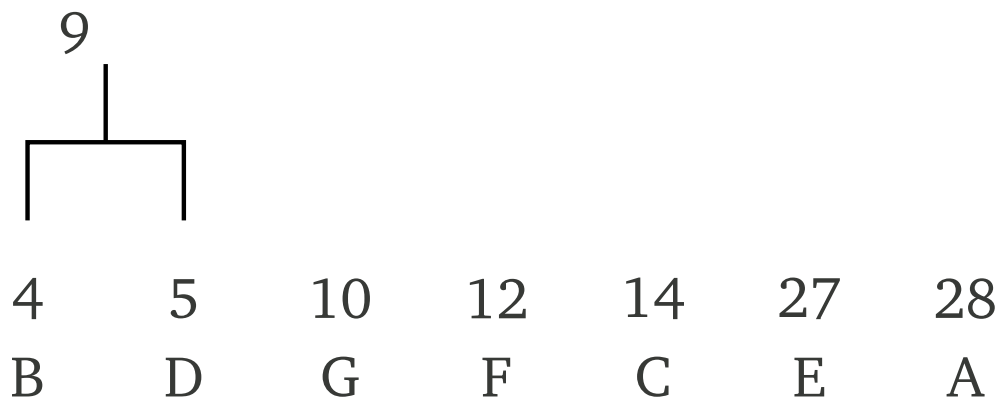
- Repeatedly select the two smallest weights and fuse them
- Another example of a greedy method.

The resulting tree is a Huffman tree.

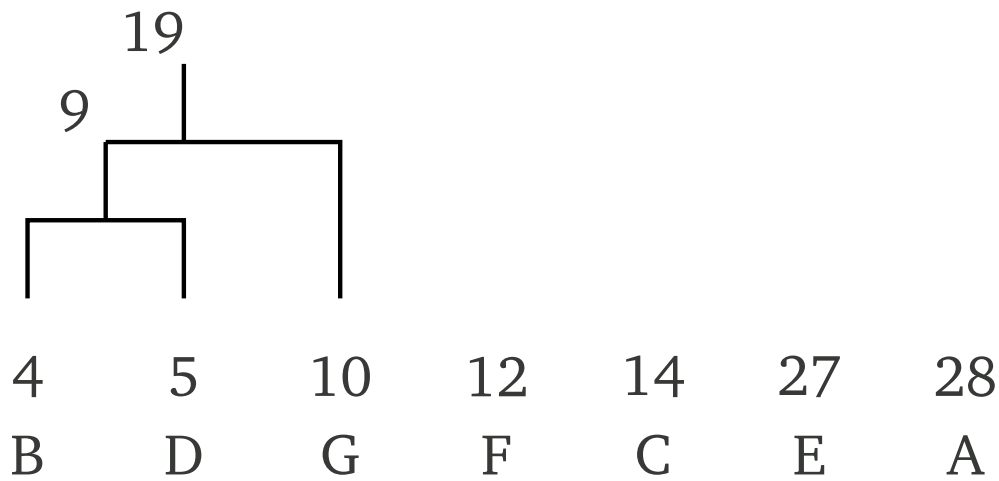
Huffman Trees

4	5	10	12	14	27	28
B	D	G	F	C	E	A

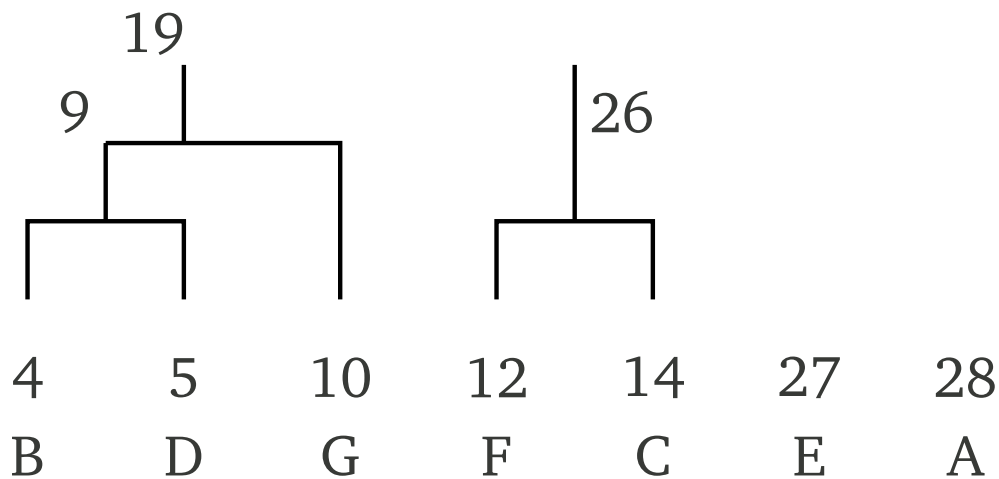
Huffman Trees



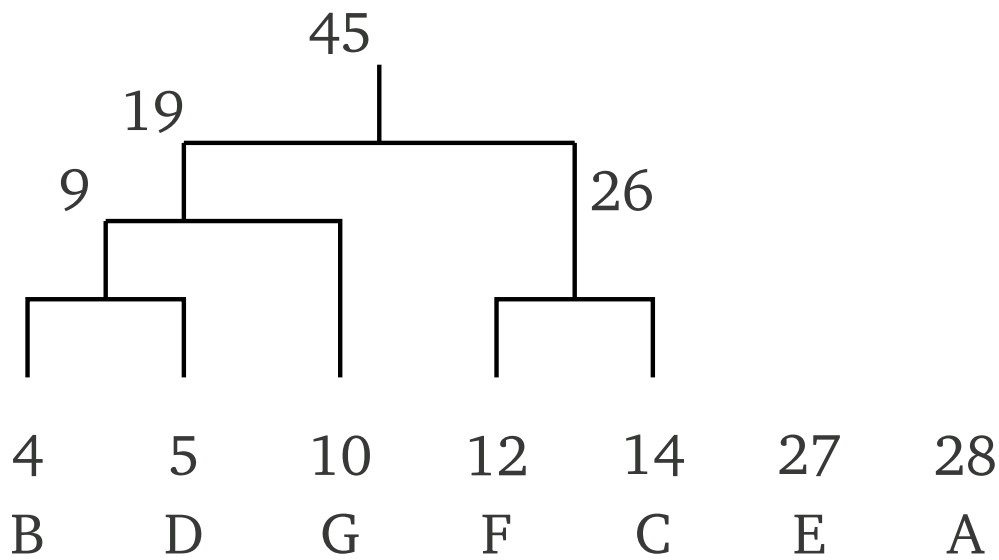
Huffman Trees



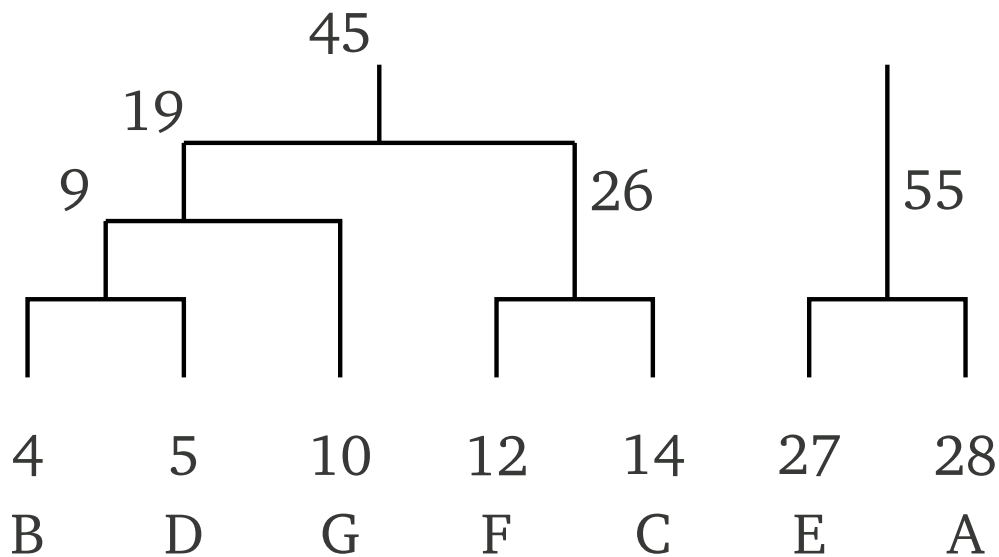
Huffman Trees



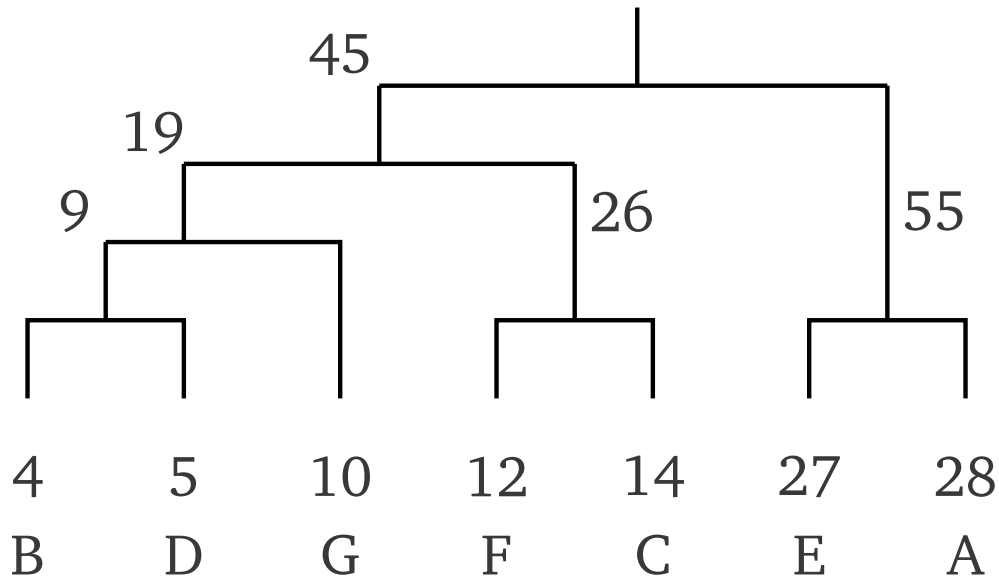
Huffman Trees



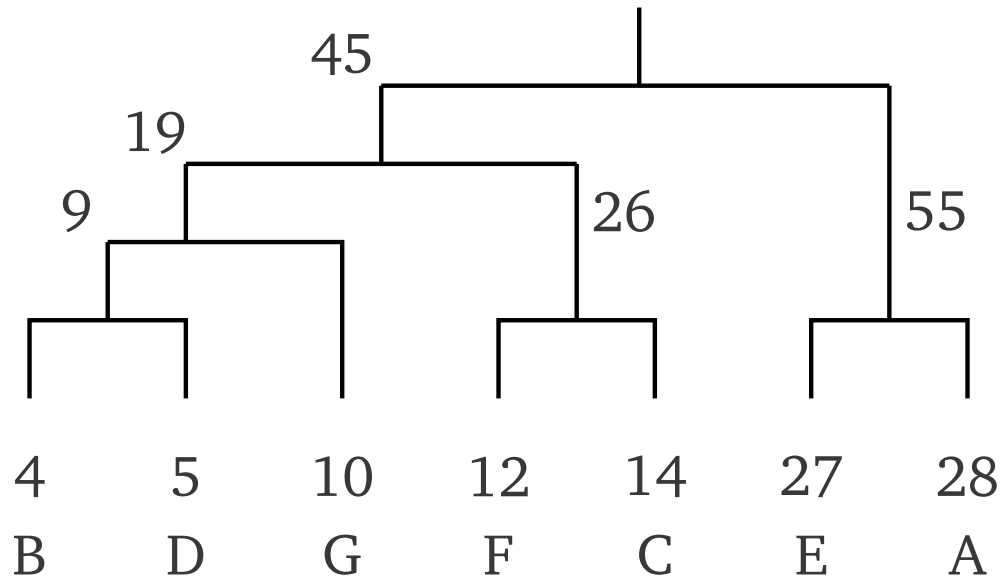
Huffman Trees



Huffman Trees



Huffman Trees



We end up with the trie from before!

One can show that this gives the best encoding.

Huffman Tree Book-Keeping

How to store trees to fuse next

Huffman Tree Book-Keeping

How to store trees to fuse next

A priority queue—a min-heap of weights.

Huffman Tree Book-Keeping

How to store trees to fuse next

A priority queue—a min-heap of weights.

Turn this into an initial min heap:

(28,A), (4,B), (14,C), (5,D), (27,E), (12,F), (10,G)

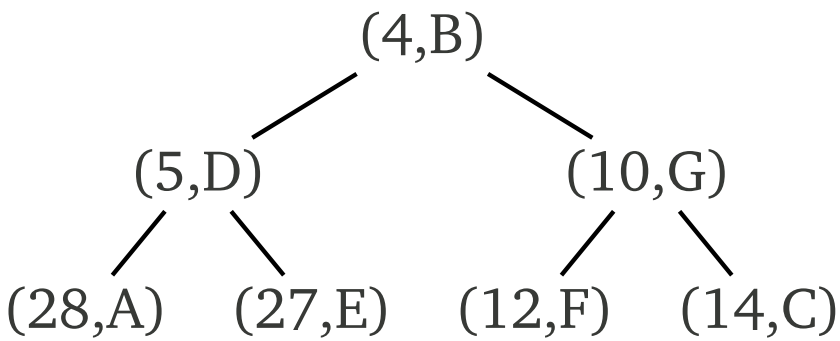
Huffman Tree Book-Keeping

How to store trees to fuse next

A priority queue—a min-heap of weights.

Turn this into an initial min heap:

(28,A), (4,B), (14,C), (5,D), (27,E), (12,F), (10,G)



while $|PQ| > 1$ **do**
  $T_1 \leftarrow \textsc{EjectMin}(PQ)$
  $T_2 \leftarrow \textsc{EjectMin}(PQ)$
  $\textsc{Inject}(\textsc{Fuse}(T_1, T_2), PQ)$

Now, while the priority queue has size 2 or more, take out the two highest-priority trees, fuse them, and put the fused tree back in. After that, a single $\textsc{EjectMin}(PQ)$ yields the Huffman tree.



Compressed Transmission

Compressed Transmission

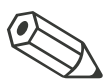
Compressed file sent from one party to another

- the parties must agree about the codes used
- or the trie can be sent along with the message
- for long files this extra cost is negligible

Modern variants of Huffman encoding

- e.g. Lempel-Ziv compression
- assign codes not to individual symbols but to sequences of symbols.

Is Huffman encoding always good?



Overview

Next Lecture

- complexity theory (a large topic)
- NP-completeness